IDENTIFICACIÓN DE CÓDIGOS DE CADA LENGUAJE DE PROGRAMACIÓN WEB

UF2405 · UD3 · Actividad 1
Ejercicio teórico-práctico

Respuesta razonada basada en el proyecto presentado
Versión 1.0 · Octubre 2026

1. Objetivo del ejercicio

Identificar el lenguaje al que pertenece cada fragmento de código, justificar la identificación mediante sus rasgos de sintaxis y señalar su entorno de ejecución habitual y el resultado esperado.

Fragmento 1 — JSP (JavaServer Pages)

<%= new java.util.Date() %>

Justificación: Se identifica por los delimitadores de expresión JSP <%= ... %>. Dentro se ejecuta una expresión Java que obtiene la fecha actual. Se procesa en el servidor Java y el resultado se inserta en la respuesta HTML.

Entorno de ejecución y extensión: Servidor; extensión habitual .jsp.

Resultado esperado: Imprime una representación de la fecha actual.

Fragmento 2 — Python

```
def dibujar_muneco(opcion):
  ...
```

Justificación: La palabra reservada def declara una función; los dos puntos abren el bloque y la indentación define su cuerpo. Si utiliza Tkinter, la parte gráfica se ejecuta en un entorno Python con interfaz gráfica, no directamente en el navegador.

Entorno de ejecución y extensión: Servidor o entorno local; extensión .py.

Resultado esperado: La función puede dibujar una figura según el valor de opcion; el resultado exacto depende del código completo.

Fragmento 3 — PHP

<?php $mensaje = "Hola"; echo $mensaje; ?>

Justificación: Los delimitadores <?php ... ?> identifican código PHP. Las variables comienzan por $, la asignación usa = y echo envía texto a la salida. Habitualmente el servidor procesa PHP antes de entregar la respuesta al navegador.

Entorno de ejecución y extensión: Servidor; extensión .php.

Resultado esperado: Genera el texto Hola.

Fragmento 4 — JavaScript

<script type="text/javascript"> ... </script>

Justificación: La etiqueta HTML script indica que el contenido incluido es JavaScript o que se carga desde un archivo. Las instrucciones pueden responder a eventos, mostrar una alerta o escribir mensajes en la consola. En el proyecto descrito se ejecuta en el navegador.

Entorno de ejecución y extensión: Cliente/navegador; extensión externa habitual .js.

Resultado esperado: Puede mostrar un mensaje de bienvenida y mensajes de diagnóstico.

Fragmento 5 — HTML

<html><head>...</head><body>...</body></html>

Justificación: Se reconoce por las etiquetas de apertura y cierre, como html, head y body. HTML es un lenguaje de marcado que define la estructura del documento; no es por sí mismo un lenguaje de programación de propósito general.

Entorno de ejecución y extensión: El navegador interpreta el marcado; extensión .html o .htm.

Resultado esperado: Organiza el contenido que el navegador representa visualmente.

Fragmento 6 — ASP clásico / VBScript

<% If Request.Form("tab") = "" Then ... %>

Justificación: Los delimitadores <% ... %> indican código de servidor ASP clásico. Request.Form recupera un valor enviado mediante un formulario y la estructura If ... Then ... End If corresponde a la sintaxis de VBScript. Su entorno histórico habitual es Microsoft IIS.

Entorno de ejecución y extensión: Servidor IIS; extensión .asp.

Resultado esperado: Evalúa si el campo tab está vacío y ejecuta la rama correspondiente; el resultado exacto depende del código completo.

2. Tabla comparativa

N.º	Lenguaje	Indicadores de sintaxis	Ejecución habitual	Extensión
1	JSP	<%= ... %>; expresión Java	Servidor	.jsp
2	Python	def, dos puntos e indentación	Servidor/local	.py
3	PHP	<?php ?>, $, echo	Servidor	.php
4	JavaScript	Etiqueta script e instrucciones	Cliente	.js
5	HTML	Etiquetas html, head, body	Navegador interpreta el marcado	.html
6	ASP/VBScript	<% %>, Request.Form, If...Then	Servidor IIS	.asp

3. Diferencia entre cliente y servidor

Cliente: JavaScript suele ejecutarse en el navegador; HTML se interpreta para construir la estructura de la página. HTML aporta marcado y JavaScript aporta comportamiento interactivo.

Servidor: JSP, PHP y ASP clásico se procesan en un servidor compatible. El navegador recibe el resultado generado, no el código fuente original de esos lenguajes. Python puede utilizarse en aplicaciones web de servidor o en entornos locales; no debe clasificarse como lenguaje de navegador en este ejemplo.

La clasificación señala el entorno habitual del fragmento concreto, no una limitación absoluta de cada lenguaje o tecnología.

4. Relación con el proyecto presentado

La aplicación organiza los ejemplos en tarjetas con el código, identificación del lenguaje, entorno de ejecución, extensión, justificación y simulación del resultado. La vista principal se describe como index.jsp; el controlador IdentificacionServlet.java reenvía la petición a la vista; estilos.css define la presentación y script.js aporta interactividad. Maven gestiona la compilación y Tomcat procesa JSP y servlets.

5. Conclusión

Los fragmentos se distinguen por sus delimitadores y construcciones sintácticas: JSP usa expresiones <%= %>; Python utiliza def e indentación; PHP emplea <?php ?>, variables con $ y echo; JavaScript se incluye con script; HTML utiliza etiquetas de marcado; y ASP clásico con VBScript combina <% %>, Request.Form e instrucciones If...Then. Reconocer estos indicios permite identificar el lenguaje y comprender dónde se procesa.